

Department of Software Engineering

Mini Weather Station

*Real-Time Temperature and Humidity Monitoring using Arduino UNO,
DHT11, and I2C LCD*

Group Members

Emaan Binte Shams — 25k-3082

Wajiha Nazir — 25k-3021

Ilham Fatima — 25k-3024

Marium Jamshed— 25k-3073

Date of Submission: 5/4/26

Supervisor

Muhammad Nadeem Ghouri

Course

Digital Logic Design Lab

Department

Department of Software Engineering

Executive Summary

Overview: This project, titled “Mini Weather Station,” is a real-time embedded system designed to measure ambient temperature and humidity using a DHT11 sensor and display the values on a 16x2 I2C LCD screen, all controlled by an Arduino UNO. The main tasks involved circuit design, sensor interfacing, programming the microcontroller in C/C++, and testing the system for stable continuous operation. The project successfully demonstrates how a low-cost microcontroller can be used to build a basic environmental monitoring device.

Key Findings: The system reliably reads temperature and humidity values once per second and updates the LCD without flicker. We learned how to integrate sensors using digital communication, how to use the I2C protocol to reduce wiring complexity, and how to debug hardware faults such as a burnt sensor and an incorrect I2C address. The final outcome is a working, demonstrable weather station that can easily be extended into an IoT-based device.

Introduction

Background: Digital Logic Design (DLD) focuses on the principles of digital systems, signal flow, and the integration of logic-based components into functional circuits. A weather station is a strong DLD-relevant project because it brings together digital sensors, microcontrollers, digital communication protocols (I2C and single-wire digital), and a digital display unit. It demonstrates how digital data flows from a sensing element through a controller to a user interface, which is the foundation of most modern embedded systems.

Project Objectives: The objectives outlined in our project proposal were:

- To design and build a working circuit using Arduino UNO, DHT11, and an I2C LCD.
- To program the Arduino in C/C++ to read sensor values and display them in real time.
- To understand and apply the I2C communication protocol.
- To handle real-world hardware issues such as wiring faults and faulty sensors.
- To prepare the project for a viva demonstration with confident technical understanding.

Project Description

Scope: The project includes a fully functional hardware setup consisting of an Arduino UNO, a DHT11 temperature/humidity sensor, and a 16x2 I2C LCD, powered through a USB cable. The system continuously reads the environment and updates the LCD every second. The scope is intentionally limited to local, real-time display — features such as cloud upload, mobile notifications, weather forecasting, and additional sensors (rainfall, pressure) are explicitly excluded from this version, but are noted as possible future extensions.

Technical Overview: The following tools and technologies were used in the project:

- Arduino UNO (ATmega328P microcontroller) as the central processing unit.
- DHT11 sensor for temperature and humidity acquisition through a digital single-wire protocol.
- I2C LCD (16x2) for output display, communicating over only two lines (SDA and SCL).
- Arduino IDE for writing, compiling, and uploading the C/C++ firmware.
- DHT.h library for sensor communication and LiquidCrystal_I2C.h library for the LCD.
- Proteus / breadboard prototyping for circuit design and verification.

Methodology

Approach: The project was completed using a weekly milestone-based plan. In the first week, we studied the datasheets of the DHT11 and the I2C LCD and finalized the circuit design. In the second week, we assembled the circuit on a breadboard and confirmed the wiring with a multimeter. In the third week, we wrote and tested the Arduino code, including library setup, sensor reading, and LCD output. In the final week, we focused on debugging, error correction, performance testing, and viva preparation.

Roles and Responsibilities: Each member of the group was assigned specific responsibilities to ensure equal participation:

Group Member	Responsibilities
Emaan binte shams	Circuit design, wiring of DHT11 and I2C LCD with Arduino UNO.
Wajiha nazir	Arduino C/C++ code: sensor reading logic, library integration, and LCD output.
Ilham Fatima	Testing, debugging, troubleshooting wiring and I2C address issues.
Marium Jamshed	Documentation, report writing

Project Implementation / Circuit Diagram

Design and Structure: The circuit is organized around the Arduino UNO, which acts as the central controller. The DHT11 sensor is connected to a 5V supply, ground, and Digital Pin 2 for data. The I2C LCD is connected to 5V, ground, and the dedicated I2C pins (A4 = SDA, A5 = SCL) on the Arduino. The breadboard is used to share power and ground rails between modules. The complete pin mapping is shown in the table below.

Components Used

S. No.	Component	Purpose / Role in Project
1	Arduino UNO	Main microcontroller board (ATmega328P) that reads sensor data and drives the LCD.
2	DHT11 Sensor	Measures ambient temperature and relative humidity using a digital single-wire protocol.
3	I2C LCD (16x2)	Displays the temperature and humidity values to the user.
4	Jumper Wires	Provide electrical connections between the Arduino, sensor, and LCD.
5	Breadboard	Used for prototyping the circuit without soldering.
6	USB Cable / 5V Supply	Powers the Arduino and connected modules.

Wiring / Pin Configuration

Module	Pin / Signal	Arduino UNO Pin
DHT11	VCC	5V
DHT11	GND	GND
DHT11	DATA	Digital Pin 2
I2C LCD	VCC	5V
I2C LCD	GND	GND
I2C LCD	SDA	A4 (SDA)
I2C LCD	SCL	A5 (SCL)

Functionalities Developed

- Real-time reading of temperature in degrees Celsius from the DHT11 sensor.
- Real-time reading of relative humidity (%) from the DHT11 sensor.
- Display of temperature on Line 1 and humidity on Line 2 of the I2C LCD.
- Continuous update loop that refreshes the readings every one second.
- Optional debug output through the Arduino Serial Monitor for testing.
- Use of the I2C protocol to keep the wiring minimal and the circuit clean.

Code

The following Arduino code demonstrates the core functionality of our temperature and humidity monitoring system. It interfaces with a DHT11 sensor to collect environmental data and uses an I2C LCD display to present the readings. The program follows a simple sequence of operations: initialize devices → read sensor data → display values on the LCD and Serial Monitor → delay → repeat continuously.

```
//Humidity sensor

#include <DHT.h>
#include <LiquidCrystal_I2C.h>
#include <Wire.h>

LiquidCrystal_I2C lcd(0x27,16,2); // set the LCD address to 0x27 for a 16
chars and 2 line display

//Constants
#define DHTPIN 2      // what pin we're connected to
#define DHTTYPE DHT11 // DHT 11
DHT dht(DHTPIN, DHTTYPE); //// Initialize DHT sensor for normal 16mhz Arduino

//Variables
//int chk;
int h; //Stores humidity value
int t; //Stores temperature value
```

```

void setup()
{
    Serial.begin(9600);
    Serial.println("Temperature and Humidity Sensor Test");
    dht.begin();
    lcd.init(); //initialize the lcd
    lcd.backlight(); //open the backlight
}

void loop()
{
    //Read data and store it to variables h (humidity) and t (temperature)
    // Reading temperature or humidity takes about 250 milliseconds!
    h = dht.readHumidity();
    t = dht.readTemperature();

    //Print temp and humidity values to serial monitor
    Serial.print("Humidity: ");
    Serial.print(h);
    Serial.print(" %, Temp: ");
    Serial.print(t);
    Serial.println(" ° Celsius");

    // set the cursor to (0,0):
    // print from 0 to 9:

    lcd.setCursor(0,0);
    lcd.print("Temperature:");
    lcd.print(t);
    lcd.print("C");

    lcd.setCursor(1, 1);
    lcd.print("Humidity:");
    lcd.print(h);
    lcd.print("%");

    delay(1000); //Delay 1 sec.
}

```

Challenges Faced

- **Incorrect wiring:** Initial connections were made without referring to the datasheet, which caused the LCD to remain blank.
- **Burnt DHT11 sensor:** Reverse polarity on VCC and GND damaged the first sensor and it stopped responding.
- **Wrong I2C address:** The default address (0x27) did not match our LCD module, so the display showed nothing initially.
- **Library conflicts:** Multiple versions of the LiquidCrystal_I2C library caused compile errors until we used a single, consistent version.

- **Unstable readings:** When the loop ran without delay, the DHT11 returned NAN values because it was being read faster than its sampling rate.

Each of these challenges was resolved through systematic debugging:

- Re-verifying every wire against the datasheet pinout.
- Replacing the burnt sensor and double-checking VCC and GND before powering up.
- Running an I2C scanner sketch to detect the correct LCD address (0x3F in our case).
- Removing duplicate libraries and reinstalling a clean version.
- Adding a 1-second delay between sensor reads to respect the DHT11 sampling limit.

Results

Project Outcomes: The Mini Weather Station meets every objective set in the proposal. The Arduino successfully reads temperature and humidity values from the DHT11 in real time, and the I2C LCD displays them in a clean, readable format. The system is stable, runs continuously without crashes, and is powered through a single USB connection. We also gained a solid practical understanding of microcontroller programming, digital communication protocols, and hardware debugging.

Screenshots and Illustrations: The accompanying demonstration includes the following visuals (to be attached during submission):

- A photograph of the final circuit on the breadboard with all components connected.
- A schematic / circuit diagram showing the connections between the Arduino, DHT11, and the I2C LCD.
- A close-up of the LCD displaying live temperature and humidity readings.
- A screenshot of the Arduino Serial Monitor showing the output stream for verification.

Testing and Validation: We tested the project under different conditions to validate its accuracy and stability. The sensor was placed near a warm object (a hand or a lamp) and the temperature reading rose appropriately. Breathing close to the sensor caused the humidity reading to spike, confirming responsiveness. The system was left running for over an hour to verify that it did not freeze, the LCD did not flicker, and the values continued to update once per second as designed.

Conclusion

Summary of Findings: The Mini Weather Station project successfully demonstrates how a microcontroller, a digital sensor, and a display can work together to monitor real-world environmental conditions. It solves the original problem of building an affordable, easy-to-use device for live temperature and humidity monitoring, and reinforces every concept from the DLD lab — digital signals, communication protocols, and integrated digital systems. All group members contributed equally and gained a strong understanding of the underlying hardware and software.

Final Remarks: Beyond the immediate result, this project lays a strong foundation for future work. With the addition of an ESP8266 / ESP32 module, the system can be upgraded into a full IoT weather station that uploads readings to the cloud. Adding a rainfall sensor or a BMP280 pressure sensor would allow basic weather forecasting features. We recommend that future groups carefully verify the I2C address of their specific LCD module and always cross-check

power connections before powering on, as these were the most common sources of issues during our build.